AI Coaching

CSE 360 Team Project Phase 2

Purpose

This document is our plan for adding AI coaching to the Lessons Learned input fields. It covers five fields: Title, Description, Root Cause, Recommendation, and Keywords. The goal is to go past basic input validation (length, allowed characters) and help the Contributor write a lesson that is actually useful to the next person who reads it.

This is a plan for TP3. None of the coaching described here is implemented in TP2. The GUI images are mockups based on our current Contributor Lessons Learned page (guiLessons.ViewLessons), and each one is labeled as a mockup.

All five sections use the same example, which is a real problem our team had in TP2: after the testing2 merge, the lesson tests failed with "Table LESSONDB not found", but only on a computer that had never run the program before.

Rules That Apply to Every Field

1. Coaching only runs when the Contributor clicks Check with AI. It does not run while typing or on save.
2. The AI never changes the Contributor's text by itself. The text only changes if the Contributor clicks Apply Suggestion. Keep Original leaves it as typed.
3. Applying a suggestion does not save the lesson. Saving still only happens through Create or Update.
4. Normal validation (LessonValidator and LessonManager) still runs on the text before it is saved. AI output never writes to the database directly.
5. If the AI needs a fact it does not have, it asks a question or leaves a [ ] placeholder. It does not guess.
6. If the AI service is slow or down, the Contributor gets a message and Create, Update, and Delete still work normally.
7. A privacy note is shown before text is sent to an outside service. A local model is preferred so the text stays on the computer.
8. A grammar checker can only say if a sentence is written well. It cannot tell if a root cause or recommendation makes sense. Those checks use a language model and are shown as advice only.

Prioritized List of Input Fields

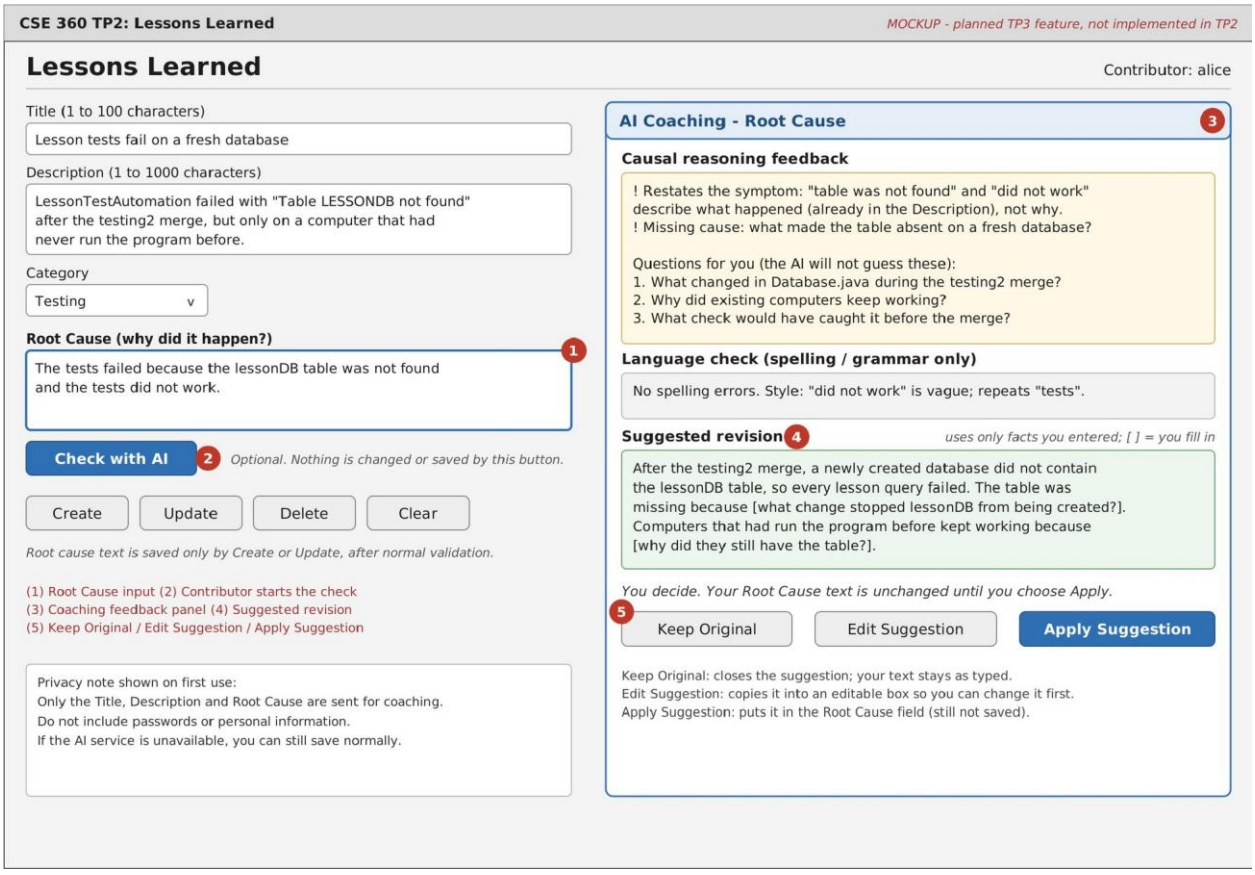
The fields are ranked by how much coaching improves the lesson for the next reader. Each field is assigned to one team member so the work is split evenly (one field each).

1. Root Cause - Levin Shibu - High (P1). If the cause just repeats the symptom, the lesson cannot help anyone prevent the problem. This is the field people get wrong the most.

2. Recommendation - Ezequiel Tarin - High (P1). This is what the next person actually does. Vague advice like "be more careful" makes the lesson useless.
3. Description - Ojas Patil - Medium (P2). The other fields depend on a clear description, but most contributors already include the basic facts.
4. Title - Harshith Vijayan - Medium (P2). The title decides if a lesson gets found and opened from the list. It is easy to fix once the description is clear.
5. Keywords - Pablo Fundora - Medium (P2). Keywords help search (requirement R14). Suggestions are helpful but optional.

1. Root Cause
Assigned to: Levin Shibu
Priority: High (P1)

GUI Mockup



1 - Root Cause text area below Category. 2 - Check with AI button. 3 - Feedback panel with causal feedback and a separate language check. 4 - Suggested revision. 5 - Keep Original / Edit Suggestion / Apply Suggestion.

Concept of Operations

1. The Contributor fills in Title, Description, Category, and a draft Root Cause, for example "The tests failed because the lessonDB table was not found and the tests did not work."
2. The Contributor clicks Check with AI. Only the Title, Description, and Root Cause are sent. The Root Cause field is not changed.
3. Two checks run. The language check looks at spelling, grammar, and wording (here "did not work" is vague and "tests" is repeated). The causal check compares the Root Cause to the Title and Description and reports if it only restates the symptom, if it uses vague phrases like "human error" or "bad code", or if it stops before explaining why. It asks "5 Whys" style questions, for example "What change stopped the table from being created?", and gives a revision that only uses facts already in the lesson. Missing facts become [ ] questions.
4. The Contributor picks Keep Original, Edit Suggestion (copies the revision into an editable box), or Apply Suggestion (puts it in the field, still not saved).
5. The Contributor can answer the questions and run Check with AI again.
6. The Contributor clicks Create or Update. The text goes through normal validation. If it fails, the normal red error shows and nothing is saved.

Other cases: if the AI is down, a message is shown and saving still works. If the Root Cause is already fine, the panel says no causal problems were found. If a [ ] placeholder is still in the text, the Contributor is warned before saving.

Example after the Contributor answers the questions: "After the testing2 merge, the statement that creates the lessonDB table was replaced, so a new database had no lessonDB table and every lesson query failed. Computers that had run the program before still had the table, which hid the problem until a fresh database was used."

Prioritized Requirements

RC-AI-01 (P1): Coaching only runs when the Contributor clicks Check with AI and never saves anything.
RC-AI-02 (P1): The Root Cause text only changes if the Contributor clicks Apply Suggestion. RC-AI-03 (P1): The coaching detects when the Root Cause just restates the symptom from the Title or Description and explains that it says what happened, not why.
RC-AI-04 (P1): The coaching points out vague causes ("did not work", generic blame, no mechanism) and quotes the phrase.
RC-AI-05 (P1): When information is missing, it asks questions and uses [ ] placeholders. It never makes up names, events, numbers, or causes.
RC-AI-06 (P1): Applied text goes through normal validation and is only saved through Create or Update.

RC-AI-07 (P1): If the AI fails or times out, the Contributor is told and can still save normally. RC-AI-08 (P2): Spelling and grammar suggestions are shown separately and labeled language only.
RC-AI-09 (P2): The revision is clearer (one cause or cause chain) but keeps the Contributor's meaning.
RC-AI-10 (P2): Edit Suggestion lets the Contributor change the suggestion before applying it. RC-AI-11 (P2): Each piece of feedback says why it was raised, so the Contributor learns the difference between a symptom and a cause.
RC-AI-12 (P2): The Contributor is warned before saving if a [ ] placeholder is left.
RC-AI-13 (P3): A guided 5 Whys mode asks one why question at a time.
RC-AI-14 (P3): A privacy note says which fields are sent. A local model can be used so text stays on the computer.

Recommended Tools

LanguageTool (free, open source) - spelling and grammar only (RC-AI-08). It has a free public API and can also be self-hosted.
Ollama with a local open model (free, open source) - the causal check (RC-AI-03, 04, 05, 09). It can compare the Root Cause to the Description, ask questions, and stay on the user's computer.
Google Gemini API free tier - backup if a computer cannot run a local model. Google says free tier content may be used to improve their products, so it should only be used with the privacy note.
5 Whys method (Lean Enterprise Institute) - not software, but the method we tell the model to follow.

Limitation: the model's feedback is advice. It can be wrong about what the root cause is, which is why the Contributor always decides.

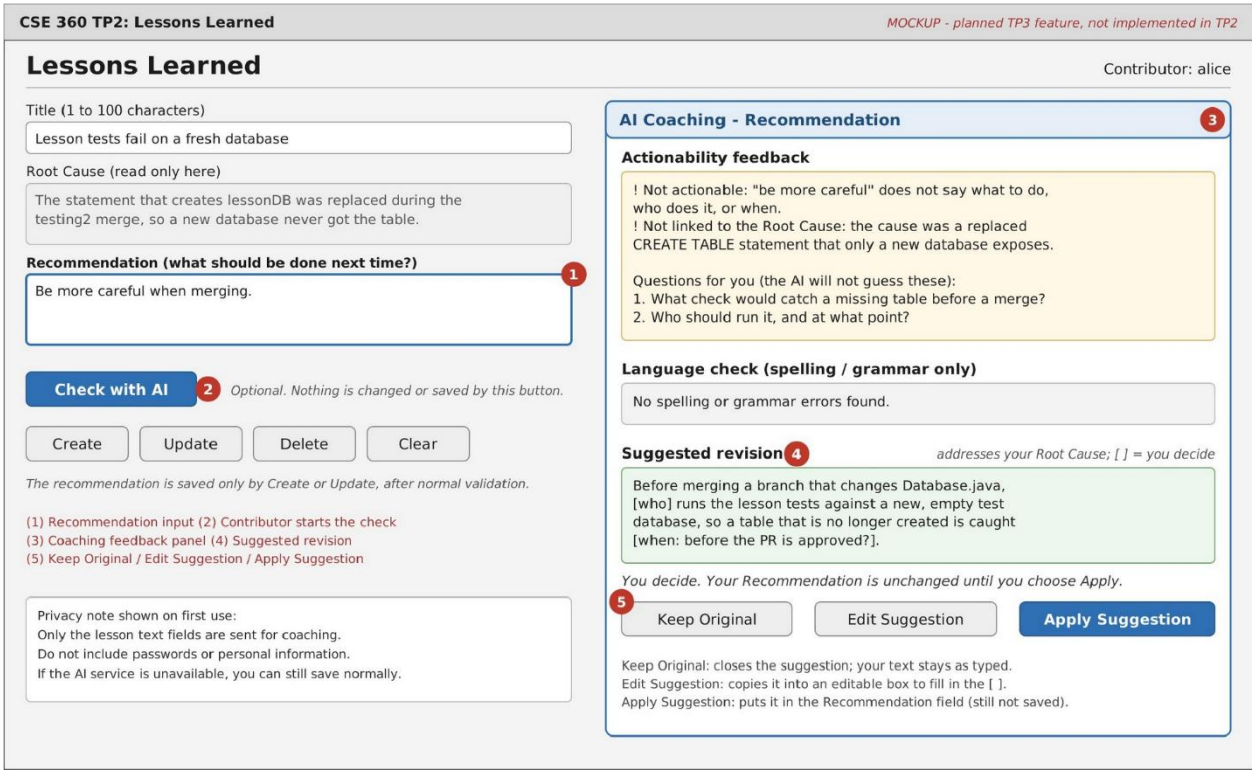
Sources

LanguageTool: https://github.com/languagetool-org/languagetool
LanguageTool public API: https://dev.languagetool.org/public-http-api
Ollama: https://github.com/ollama/ollama
Gemini API pricing: https://ai.google.dev/pricing
5 Whys: https://www.lean.org/lexicon-terms/5-whys/

2. Recommendation

Assigned to: Ezequiel Tarin

Priority: High (P1)



1 - Recommendation text area (Root Cause shown read-only above it). 2 - Check with AI. 3 - Feedback panel. 4 - Suggested revision with [ ] placeholders. 5 - Keep Original / Edit Suggestion / Apply Suggestion.

Concept of Operations

1. The Contributor types a Recommendation, for example "Be more careful when merging."
2. Check with AI sends the Title, Description, Root Cause, and Recommendation. The field is not changed.
3. LanguageTool checks the language. The model checks that the recommendation is actionable (what to do, and who and when if possible), that it would actually prevent the Root Cause, and that it is not vague advice like "test more" or "communicate better". It asks questions to make it specific. It can propose an action that follows from the Root Cause, labeled as a proposal. Things it cannot know, like who or when, are left as [ ].
4. The Contributor keeps, edits, or applies the suggestion.
5. Create or Update saves it after normal validation. The Contributor is warned about any [ ] left.

Other cases: if the Root Cause is empty, the alignment check is skipped and the Contributor is told. If there are several recommendations in the field, each one is checked. If the service is down, saving still works.

Prioritized Requirements

RE-AI-01 (P1): Coaching only runs when the Contributor clicks Check with AI and never saves.
RE-AI-02 (P1): The field only changes if the Contributor clicks Apply Suggestion.
RE-AI-03 (P1): Vague advice ("be more careful", "test more") is pointed out and quoted. RE-AI-04 (P1): The coaching checks if the recommendation addresses the Root Cause and explains any mismatch.
RE-AI-05 (P1): Things the AI cannot know (who, when, which tool) are asked or left as [ ], never made up.
RE-AI-06 (P1): Applied text is validated normally. If the AI fails, saving still works.
RE-AI-07 (P2): Any action the AI proposes must follow from facts in the lesson and is labeled as a proposal.
RE-AI-08 (P2): Spelling and grammar suggestions are shown separately.
RE-AI-09 (P2): The Contributor can edit a suggestion before applying it and is warned about [ ] before saving.
RE-AI-10 (P3): If the Root Cause is empty, the Contributor is told the alignment could not be checked.

Recommended Tools

LanguageTool (free, open source) - grammar and spelling only.
Ollama with a local open model (free, open source) - compares the Root Cause and Recommendation and spots vague advice, which a grammar checker cannot do.
5 Whys method (Lean Enterprise Institute) - a recommendation should fix the root cause found with 5 Whys, not just the symptom.

Limitation: the model does not know what the team can actually do, so it asks instead of deciding.

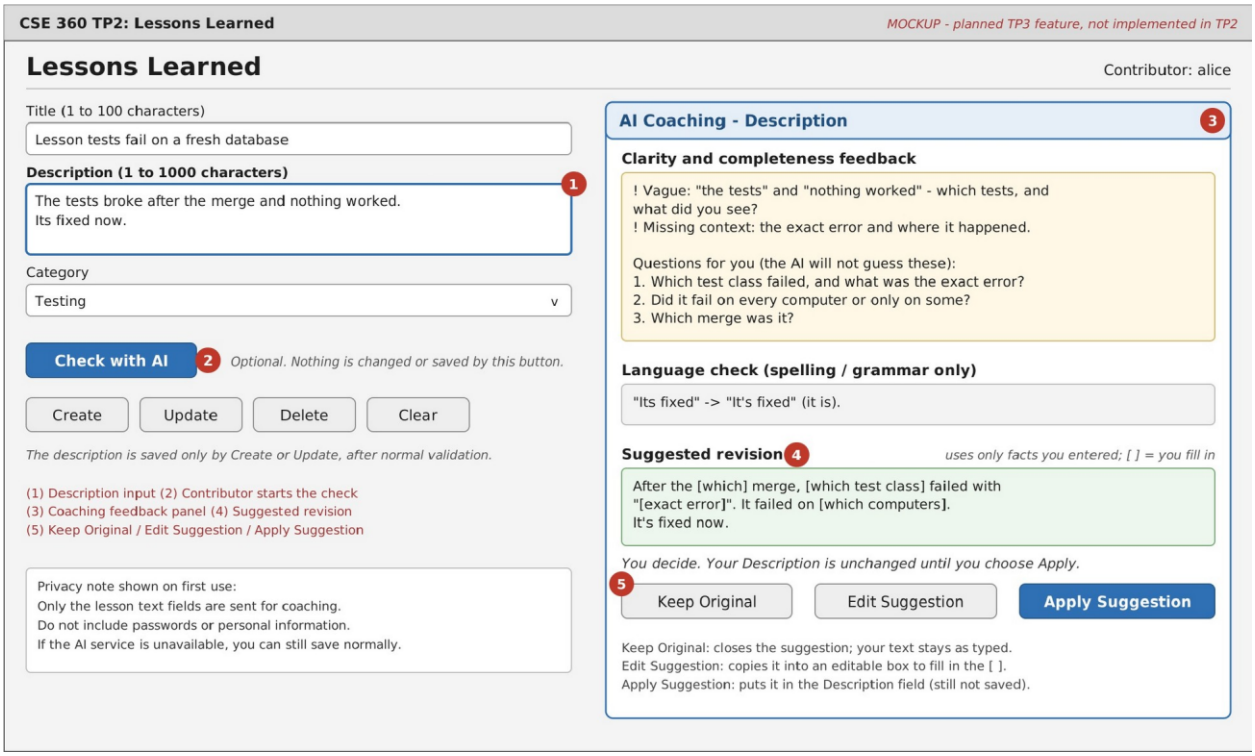
Sources

LanguageTool: https://github.com/languagetool-org/languagetool
Ollama: https://github.com/ollama/ollama
5 Whys: https://www.lean.org/lexicon-terms/5-whys/

3. Description

Assigned to: Ojas Patil

Priority: Medium (P2)



1 - Description text area. 2 - Check with AI. 3 - Feedback panel with clarity feedback, questions, and a separate language check. 4 - Suggested revision with [ ] placeholders. 5 - Keep Original / Edit Suggestion / Apply Suggestion.

Concept of Operations

1. The Contributor types a Description, for example "The tests broke after the merge and nothing worked. Its fixed now."
2. Check with AI sends the Title and Description. The field is not changed.
3. LanguageTool catches errors like "Its fixed" -> "It's fixed". The model looks for vague wording ("the tests", "nothing worked") and missing context: what happened, where, when, and what was seen (an error message or behavior). For each gap it asks a question, and it gives a revision with [ ] for each missing fact.
4. The Contributor answers the questions using Edit Suggestion, applies it, or keeps the original.
5. Create or Update saves it after the normal 1-1000 character check. The Contributor is warned if a [ ] is still in the text.

Other cases: if the Description is already clear, the panel says so. If it is over 1000 characters, the coaching suggests what to cut and the normal error still shows on save. If the service is down, saving still works.

Prioritized Requirements

DE-AI-01 (P1): Coaching only runs when the Contributor clicks Check with AI and never saves.
DE-AI-02 (P1): The field only changes if the Contributor clicks Apply Suggestion.
DE-AI-03 (P1): When context is missing, it asks specific questions and uses [ ]. It never makes up error messages, test names, dates, or events.
DE-AI-04 (P1): Applied text is checked by the normal Description rule (1-1000 characters) before saving.
DE-AI-05 (P1): If the AI fails, the Contributor is told and saving still works.
DE-AI-06 (P2): Vague wording is pointed out and the phrase is quoted.
DE-AI-07 (P2): Spelling and grammar suggestions are shown separately.
DE-AI-08 (P2): The revision keeps the Contributor's meaning and facts.
DE-AI-09 (P2): The Contributor is warned before saving if a [ ] is left.
DE-AI-10 (P3): If the Description also contains the cause or the fix, the coaching suggests moving it to Root Cause or Recommendation.

Recommended Tools

LanguageTool (free, open source) - catches common grammar and spelling mistakes in quickly written descriptions.
Ollama with a local open model (free, open source) - finds vague phrases and missing what/where/when information and writes questions instead of guessing.

Limitation: the model cannot know the facts. It can only notice they are missing, so it asks. The free LanguageTool API has rate limits, so heavy use would need a self-hosted server.

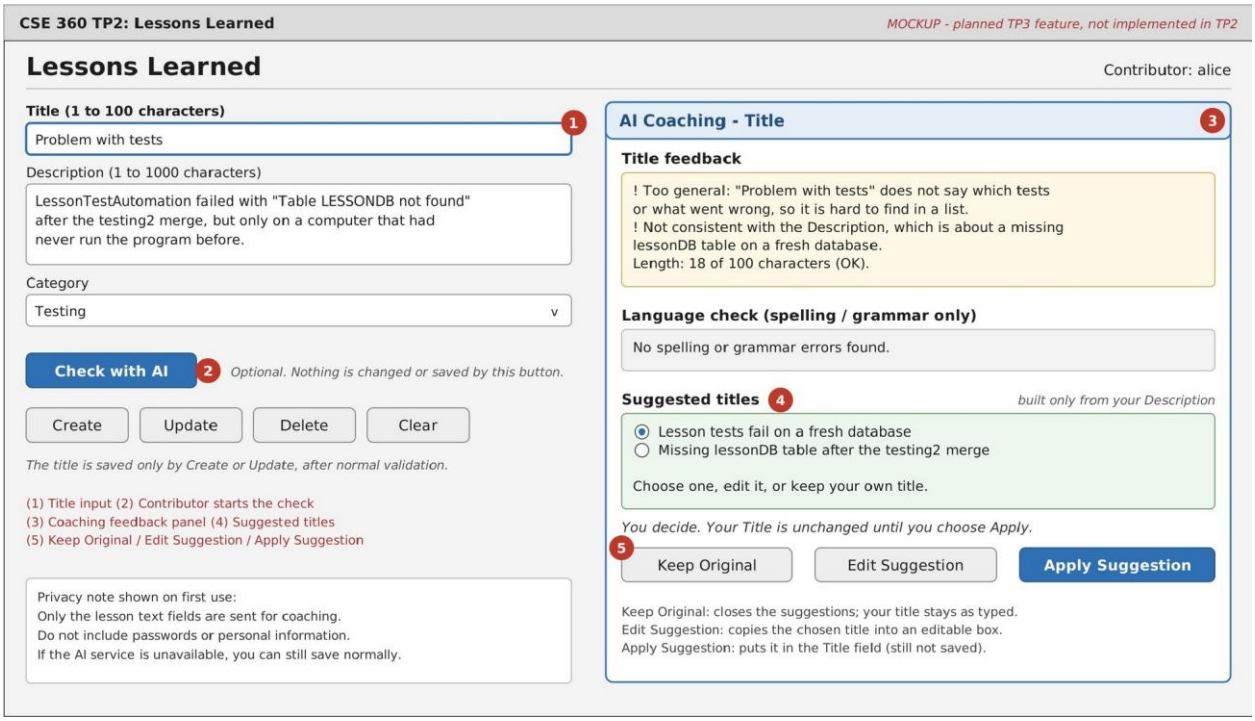
Sources

LanguageTool: https://github.com/languagetool-org/languagetool
LanguageTool public API: https://dev.languagetool.org/public-http-api
Ollama: https://github.com/ollama/ollama

4. Title

Assigned to: Harshith Vijayan

Priority: Medium (P2)



1 - Title field. 2 - Check with AI. 3 - Feedback panel with title feedback and a separate language check. 4 - Suggested titles (pick at most one). 5 - Keep Original / Edit Suggestion / Apply Suggestion.

Concept of Operations

1. The Contributor types a Title, for example "Problem with tests", plus a Description and Category.
2. Check with AI sends only the Title and Description. The Title field is not changed.
3. LanguageTool checks spelling, grammar, and capitalization. The model compares the Title to the Description and reports if the title is too general ("problem with tests", "issue"), does not match the Description, or is over 100 characters. It suggests one to three titles using only words and facts from the Description.
4. The Contributor clicks Keep Original, Edit Suggestion, or Apply Suggestion (puts the title in the field, still not saved).
5. Create or Update saves it after the normal 1-100 character check.

Other cases: if the Description is empty, the AI says it needs one first. If the title is already good, it says no changes suggested. If the service is down, saving still works.

Prioritized Requirements

TI-AI-01 (P1): Coaching only runs when the Contributor clicks Check with AI and never saves.
TI-AI-02 (P1): The Title only changes if the Contributor clicks Apply Suggestion.
TI-AI-03 (P1): Suggested titles only use facts from the Description.
TI-AI-04 (P1): Every suggested title is 1-100 characters, the same limit as LessonValidator.validateTitle. Applied titles are still validated on save.
TI-AI-05 (P1): If the AI fails, the Contributor is told and Create and Update still work.
TI-AI-06 (P2): A title that is too general is flagged with the reason (hard to find in a list).
TI-AI-07 (P2): A title that does not match the Description is flagged.
TI-AI-08 (P2): Spelling and grammar suggestions are shown separately.
TI-AI-09 (P2): The Contributor can edit a suggested title before applying it.
TI-AI-10 (P3): It warns if the title is almost the same as one of the Contributor's other lesson titles.

Recommended Tools

LanguageTool (free, open source, LGPL 2.1+) - spelling and grammar. The free public API allows 20 requests per minute and 20 KB per request, and it can be self-hosted.
Ollama with a local open model (free, open source, MIT) - compares the title to the description and suggests specific titles. Runs on the computer through a local API (http://localhost:11434).

Limitation: suggested titles are advice and can still be weak, which is why the Contributor chooses. Ollama needs a computer that can run a model. Ollama's cloud models would send text off the computer, so they should not be used without the privacy note.

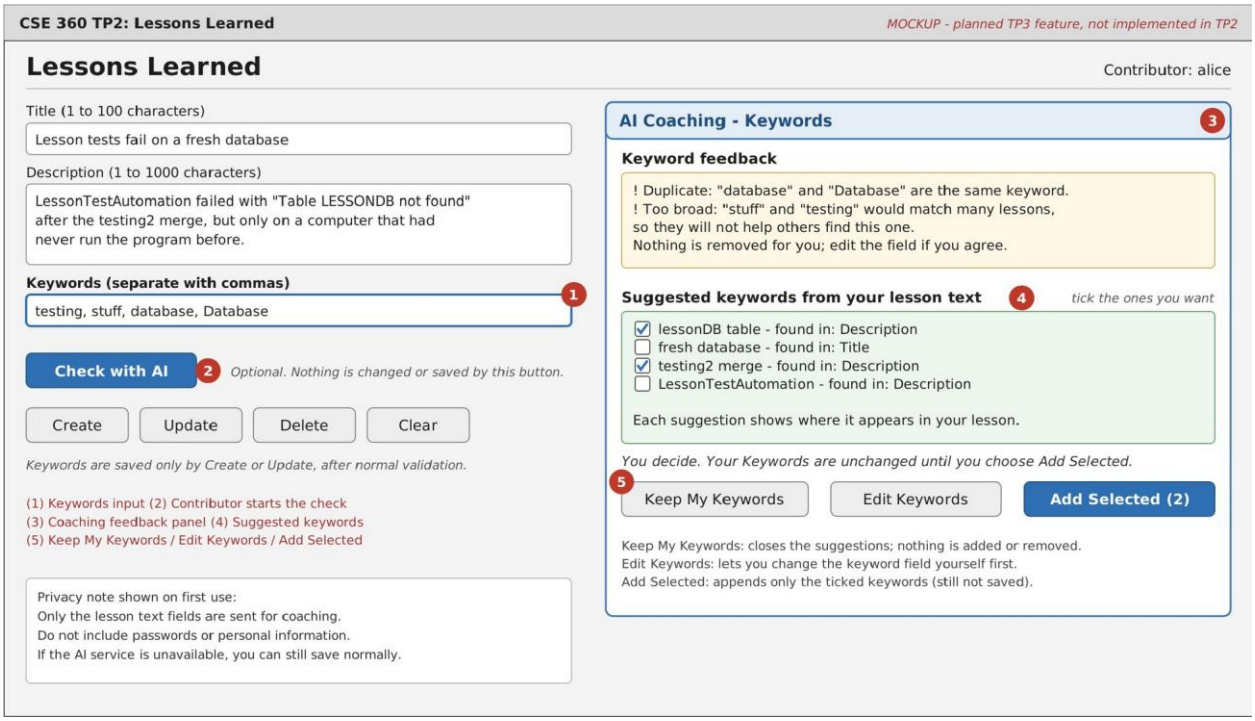
Sources

LanguageTool: https://github.com/languagetool-org/languagetool
LanguageTool public API: https://dev.languagetool.org/public-http-api
Ollama: https://ollama.com/ and https://github.com/ollama/ollama

5. Keywords

Assigned to: Pablo Fundora

Priority: Medium (P2)



1 - Keywords field (comma separated). 2 - Check with AI. 3 - Feedback panel showing duplicates and broad keywords. 4 - Suggested keywords, each showing where it was found and each ticked separately. 5 - Keep My Keywords / Edit Keywords / Add Selected.

Concept of Operations

1. The Contributor types keywords, for example "testing, stuff, database, Database".
2. Check with AI sends the Title, Description, Root Cause, Recommendation, and Keywords. The Keywords field is not changed.
3. Duplicates (same word ignoring case and spaces) are found with a simple string check, no AI needed. Very general words like "stuff" or "problem" are flagged because they will not help anyone find the lesson. KeyBERT pulls keywords and short phrases from the lesson's own text, and each suggestion shows which field it came from. Ollama can optionally give a one-line reason for each suggestion or flag.
4. The Contributor ticks the suggestions they want and clicks Add Selected. Nothing is removed automatically. Duplicates and broad words are only reported, and the Contributor deletes them by editing the field.
5. Create or Update saves the keywords after normal validation.

Other cases: if the lesson text is too short, no suggestions are made and the Contributor is told why. A suggestion that is already in the field is not shown again. If the service is down, the Contributor can still type keywords and save.

Prioritized Requirements

KW-AI-01 (P1): Coaching only runs when the Contributor clicks Check with AI and never saves.
KW-AI-02 (P1): Keywords are never added or removed automatically. Only ticked suggestions are added, and only when the Contributor clicks Add Selected.
KW-AI-03 (P1): Every suggested keyword comes from the lesson's own text and shows which field it came from.
KW-AI-04 (P1): The Keywords text is validated normally before saving. If the AI fails, keywords can still be entered and saved.
KW-AI-05 (P2): Duplicate keywords are reported but not removed.
KW-AI-06 (P2): Very broad keywords are reported with the reason.
KW-AI-07 (P2): Suggestions already in the field are not offered again.
KW-AI-08 (P3): An optional short explanation is shown for each suggestion or flag. KW-AI-09 (P3): Suggestions can include short phrases (two to three words).

Recommended Tools

KeyBERT (free, open source, MIT, Python) - a keyword extractor that ranks words and phrases from the lesson text by similarity to the whole lesson, so every suggestion comes from the lesson (KW-AI-03). It supports phrases (keyphrase_ngram_range, KW-AI-09) and more varied results (use_mmr).
Ollama with a local open model (free, open source) - optional one-line explanations.
Duplicates - no tool needed, just a string comparison.

Limitation: KeyBERT is Python and our app is Java, so it would have to run as a small local service the Java code calls. That is a TP3 design task. The first run downloads an embedding model.

Sources

KeyBERT: https://github.com/MaartenGr/KeyBERT
KeyBERT documentation: https://maartengr.github.io/KeyBERT/
Ollama: https://github.com/ollama/ollama

References

1. LanguageTool, open source proofreading software.
https://github.com/languagetool-org/languagetool
2. LanguageTool, Public HTTP Proofreading API. https://dev.languagetool.org/public-http-api
3. Ollama, official site. https://ollama.com/
4. Ollama, GitHub repository. https://github.com/ollama/ollama

5. Google AI for Developers, Gemini Developer API pricing. https://ai.google.dev/pricing
6. Google AI for Developers, Rate limits. https://ai.google.dev/gemini-api/docs/rate-limits
7. M. Grootendorst, KeyBERT repository. https://github.com/MaartenGr/KeyBERT

8. M. Grootendorst, KeyBERT documentation. https://maartengr.github.io/KeyBERT/
9. Lean Enterprise Institute, 5 Whys. https://www.lean.org/lexicon-terms/5-whys/